Mobile Phone Price Prediction Project

By: Shailendra Nogia

NEXTHIKES IT SOLUTIONS PROJECT

Introduction

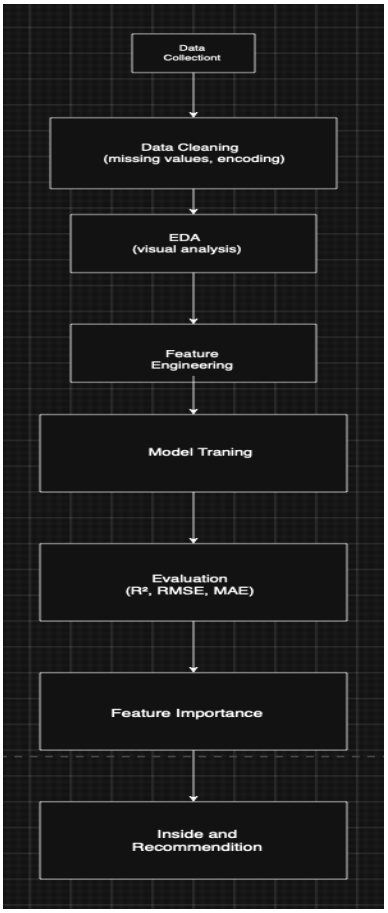
The Mobile Phone Price Prediction project aims to predict the selling price of smartphones based on their specifications such as RAM, battery capacity, internal storage, brand, display size, and other key features. The objective is to use machine learning algorithms to analyse these factors and determine how they influence mobile pricing. This analysis helps businesses set competitive prices and identify key components that impact the cost of a smartphone.

Objective

- To analyse the dataset and understand key factors influencing mobile prices.
- To preprocess and clean the data by handling missing and categorical values.
- To perform exploratory data analysis (EDA) and visualize trends.
- To apply multiple regression algorithms and compare their performance.
- To determine the most important features affecting mobile phone prices.
- To provide insights and recommendations for better pricing strategy.

Flowchart

The following flow represents the complete process followed in the project:



Python Code

- Importing Libraries

```
[1]: # STEP 1: IMPORT LIBRARIES & SETUP

     import os, warnings
     warnings.filterwarnings("ignore")

     import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt

     from sklearn.model_selection import train_test_split
     from sklearn.impute import SimpleImputer
     from sklearn.preprocessing import OneHotEncoder, StandardScaler
     from sklearn.compose import ColumnTransformer
     from sklearn.pipeline import Pipeline
     from sklearn.linear_model import LinearRegression, Ridge, Lasso
     from sklearn.tree import DecisionTreeRegressor
     from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
     from sklearn.neighbors import KNeighborsRegressor
     from sklearn.svm import SVR
     from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
     from sklearn.feature_selection import SelectKBest, f_regression

     # Try XGBoost
     try:
         from xgboost import XGBRegressor
         has_xgb = True
     except Exception:
         has_xgb = False

     OUT = "./mobile_price_outputs"
     os.makedirs(OUT, exist_ok=True)
```

- Loading and Cleaning Dataset

```
[11]: # STEP 2: LOAD DATASET  & DETECT TARGET COLUMN

      df = pd.read_csv("Processed_Flipdata.csv")
      print("Dataset shape:", df.shape)

      df.head()

      Dataset shape: (541, 12)
```

```
# Detect target column
candidates = [c for c in df.columns if any(k in c.lower() for k in ("price","prize","cost","amount","selling"))]
if candidates:
    target_col = candidates[0]
else:
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    target_col = df[numeric_cols].var().sort_values(ascending=False).index[0]

print("Detected target column:", target_col)

Detected target column: Prize
```

- Exploratory Data Analysis (EDA)

```python
# 4.2 Missing Values Check

missing = df.isnull().sum().sort_values(ascending=False)
missing = missing[missing > 0]
if len(missing) > 0:
    print("\nMissing Values Detected:\n")
    print(missing)
    plt.figure(figsize=(10,5))
    sns.barplot(x=missing.index, y=missing.values, color='tomato')
    plt.xticks(rotation=90)
    plt.title("Missing Values per Feature")
    plt.xlabel("Features")
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()
else:
    print("\nNo missing values detected.\n")
```

```python
# 4.3 Target Variable Distribution

plt.figure(figsize=(8,5))
sns.histplot(df[target_col], kde=True, bins=40, color='teal')
plt.title(f"Distribution of {target_col}")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Check skewness
skew_val = df[target_col].skew()
print(f"Skewness of {target_col}: {skew_val:.2f}")
if skew_val > 1:
    print("Strong Right Skew (few expensive phones dominate)")
elif skew_val < -1:
    print("Left Skew")
else:
    print("Fairly Normal Distribution")
```

```python
# 4.4 Univariate Analysis — Numeric Features

numeric_features = df.select_dtypes(include=[np.number]).columns.tolist()
numeric_features = [c for c in numeric_features if c != target_col]

if len(numeric_features) > 0:
    plt.figure(figsize=(14, 8))
    df[numeric_features].hist(bins=30, figsize=(14, 8), color='lightblue', edgecolor='black')
    plt.suptitle("Distribution of Numeric Features", fontsize=14)
    plt.tight_layout()
    plt.show()
```

```python
# 4.5 Correlation Heatmap (Top Numeric Features)

if len(numeric_features) > 1:
    plt.figure(figsize=(10,6))
    corr = df[numeric_features + [target_col]].corr()
    sns.heatmap(corr, annot=True, cmap="YlGnBu", fmt=".2f")
    plt.title("Correlation Heatmap with Target Variable")
    plt.tight_layout()
    plt.show()

    print("\nTop Correlations with Target:\n")
    print(corr[target_col].sort_values(ascending=False).head(10))
```

```python
# 4.6 Categorical Feature Distribution

categorical_features = df.select_dtypes(include=['object', 'category']).columns.tolist()
if len(categorical_features) > 0:
    for col in categorical_features[:5]:  # Show top 5 to avoid overload
        plt.figure(figsize=(8,4))
        df[col].value_counts().head(10).plot(kind='bar', color='mediumpurple')
        plt.title(f"Top 10 Categories in {col}")
        plt.ylabel("Count")
        plt.tight_layout()
        plt.show()
```

```python
# 4.7 Relationship between Key Features and Target

potential_features = ['RAM', 'Battery', 'Memory', 'Display', 'Storage', 'Rating', 'Camera']
for col in potential_features:
    if col in df.columns:
        plt.figure(figsize=(6,4))
        sns.boxplot(x=col, y=target_col, data=df)
        plt.title(f"{col} vs {target_col}")
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()
```

```python
# 4.8 Outlier Detection

plt.figure(figsize=(8,5))
sns.boxplot(x=df[target_col], color='orange')
plt.title(f"Outlier Detection in {target_col}")
plt.xlabel("Price")
plt.tight_layout()
plt.show()
```

```python
# 4.9 Feature Relationships (Pairplot)

if len(numeric_features) >= 3:
    sampled = df.sample(min(200, len(df)), random_state=42)
    sns.pairplot(sampled[numeric_features + [target_col]], diag_kind='kde')
    plt.suptitle("Pairwise Relationships (Sampled Data)", y=1.02)
    plt.show()

print("\nEDA Completed Successfully!")
```

- Preprocessing Pipeline using Column Transformer

```python
# STEP 7: PREPROCESSING PIPELINE

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Missing")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, num_cols),
    ("cat", categorical_transformer, cat_cols)
], remainder="drop")

print("Preprocessing pipeline created successfully!")
```

```
Preprocessing pipeline created successfully!
```

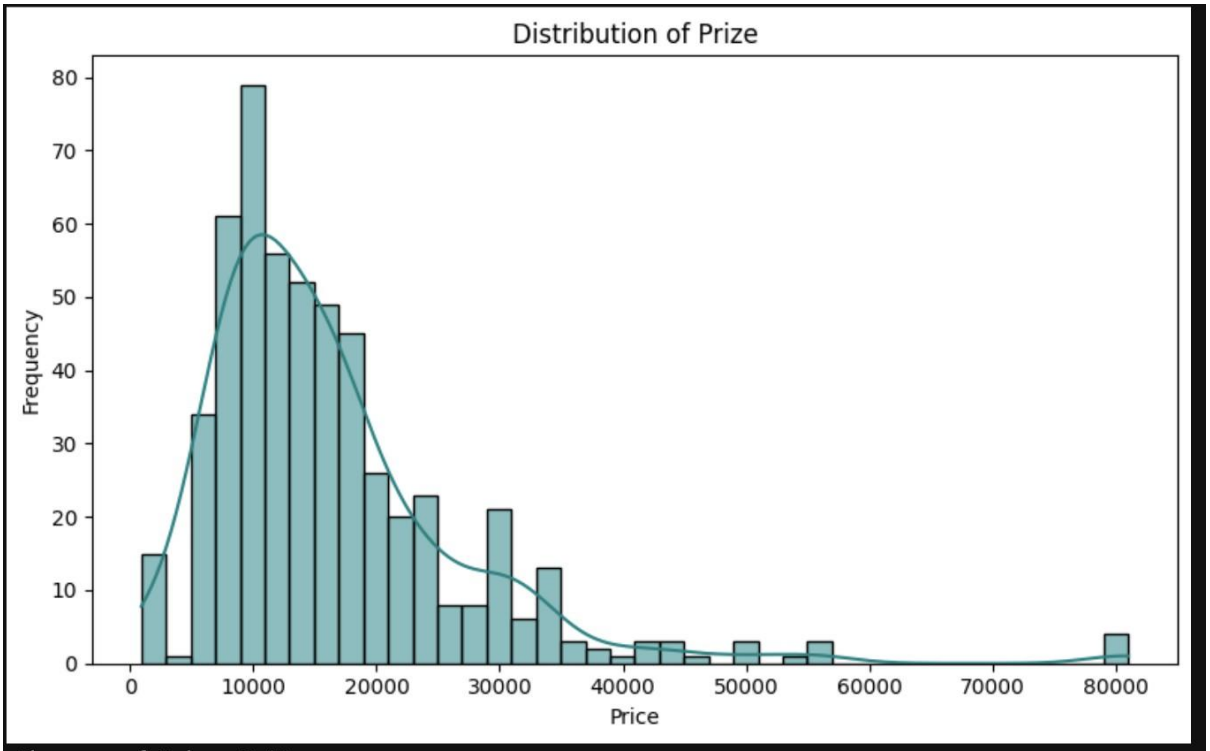
- Model Training

```python
# STEP 9: TRAIN MULTIPLE MODELS & COMPARE RESULTS

models = {
    "Linear Regression": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Lasso": Lasso(alpha=0.001),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42),
    "KNN": KNeighborsRegressor(n_neighbors=5),
    "SVR": SVR()
}

if has_xgb:
    models["XGBoost"] = XGBRegressor(n_estimators=300, learning_rate=0.05, random_state=42)

results = []
print("Training models... This may take a few minutes.\n")

for name, model in models.items():
    pipe = Pipeline([("preprocessor", preprocessor), ("model", model)])
    pipe.fit(X_train, y_train)
    preds = pipe.predict(X_test)

    try:
        rmse = mean_squared_error(y_test, preds, squared=False)
    except TypeError:
        rmse = np.sqrt(mean_squared_error(y_test, preds))

    mae = mean_absolute_error(y_test, preds)
    r2 = r2_score(y_test, preds)

    results.append({"Model": name, "R2": r2, "RMSE": rmse, "MAE": mae})
    print(f"{name:20s} | R2={r2:.4f} | RMSE={rmse:.2f} | MAE={mae:.2f}")

results_df = pd.DataFrame(results).sort_values(by="R2", ascending=False)
print("\nModel Comparison Table:")
display(results_df)
```

Screenshot of output

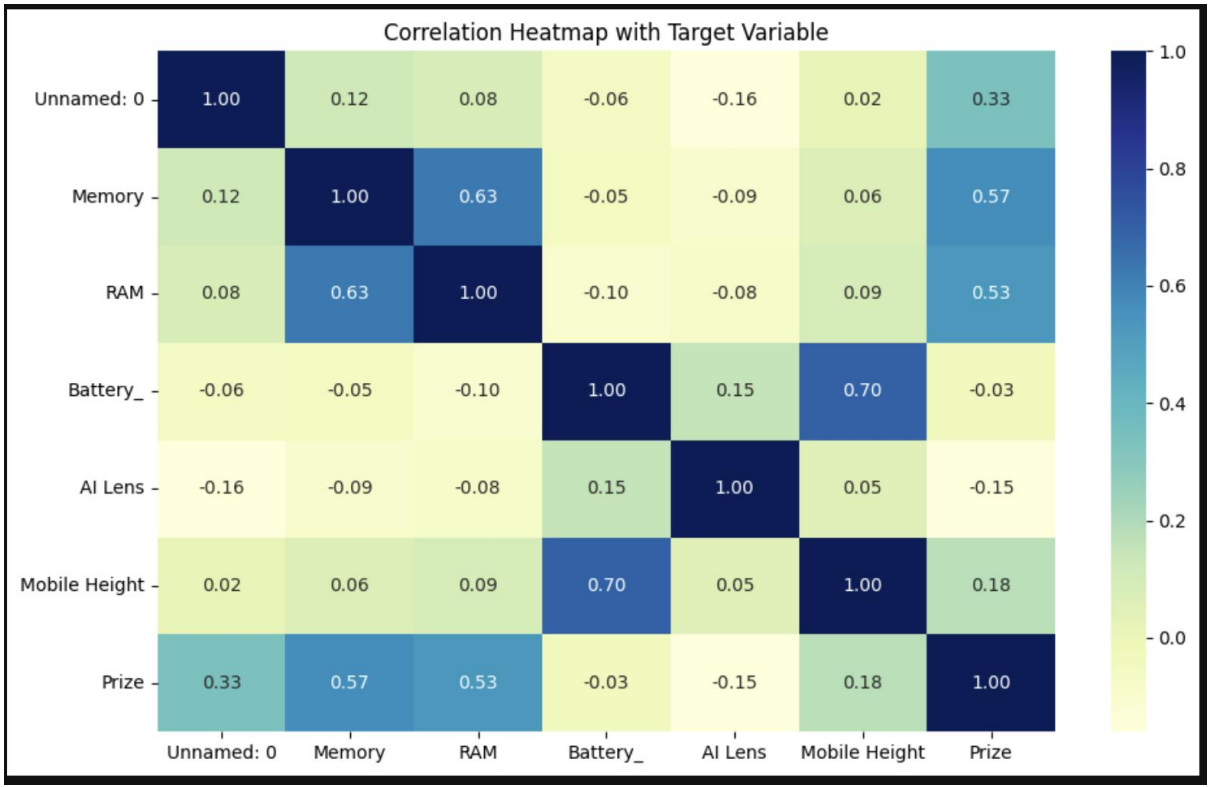
The following visualizations were generated during the project:

- Distribution of target variable (Price)

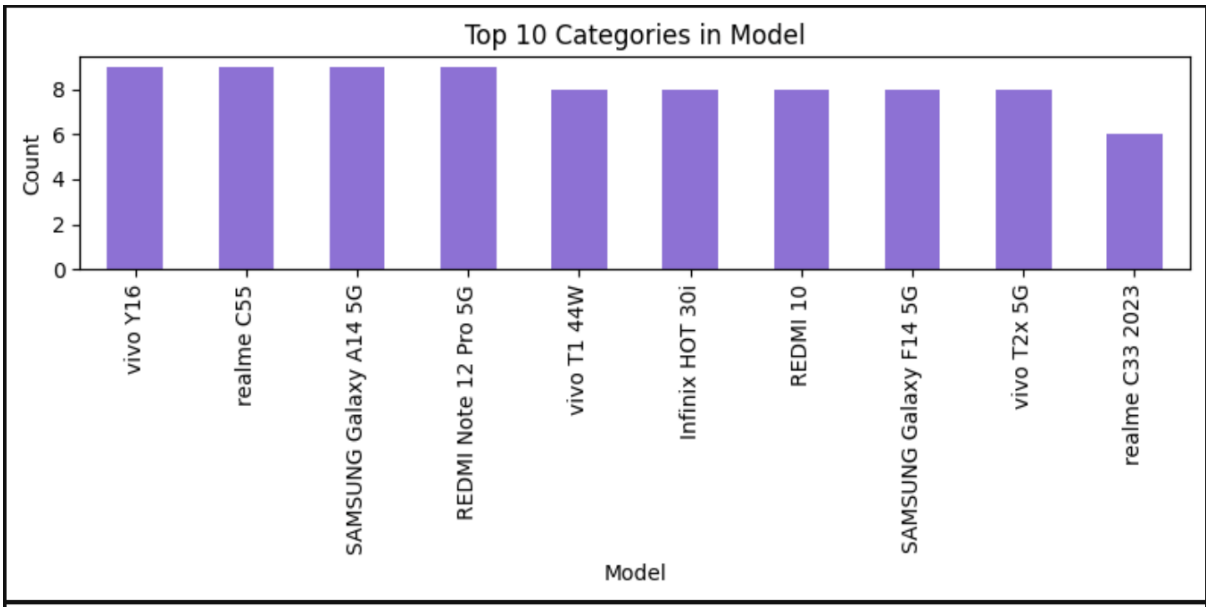


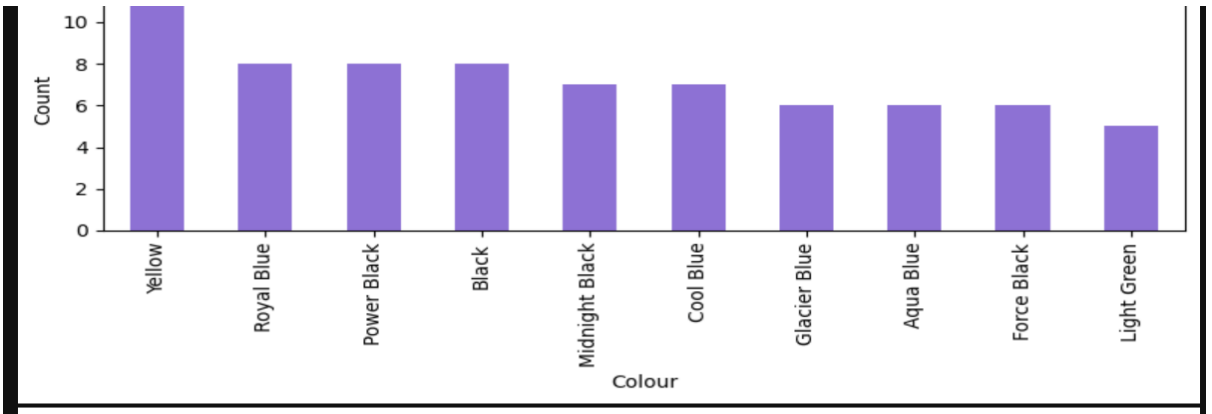

```
Skewness of Prize: 2.43
Strong Right Skew (few expensive phones dominate)
```

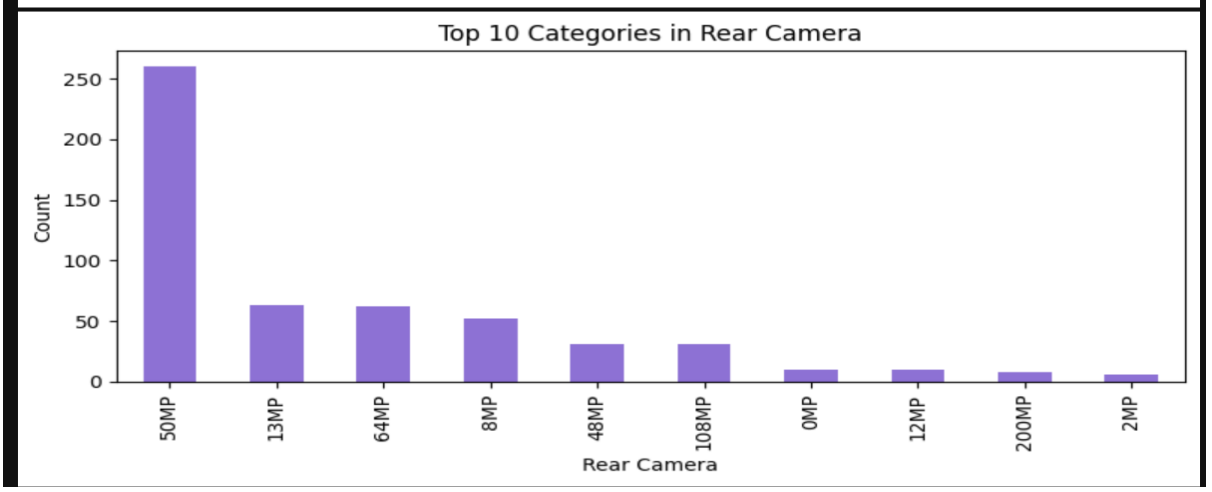
- Correlation Heatmap

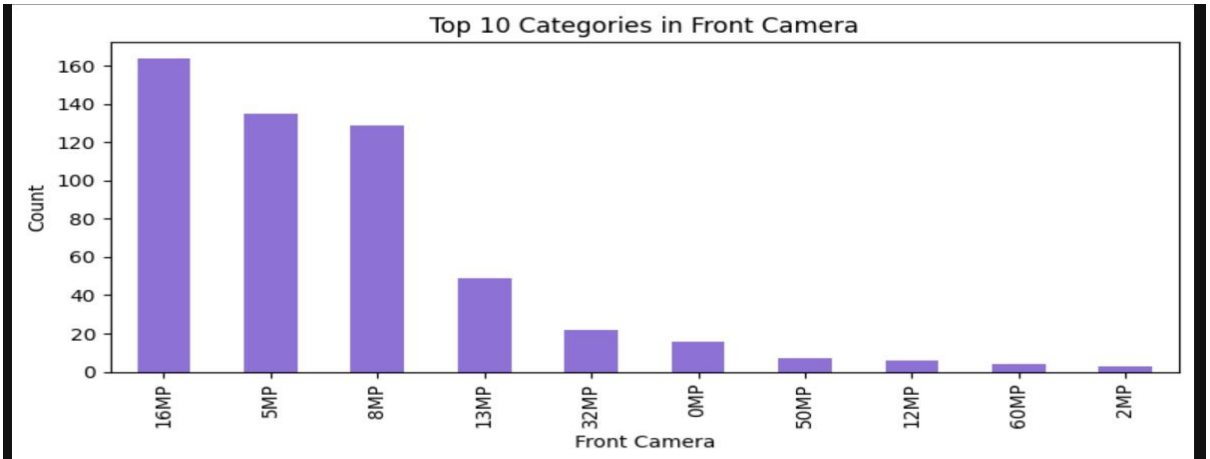


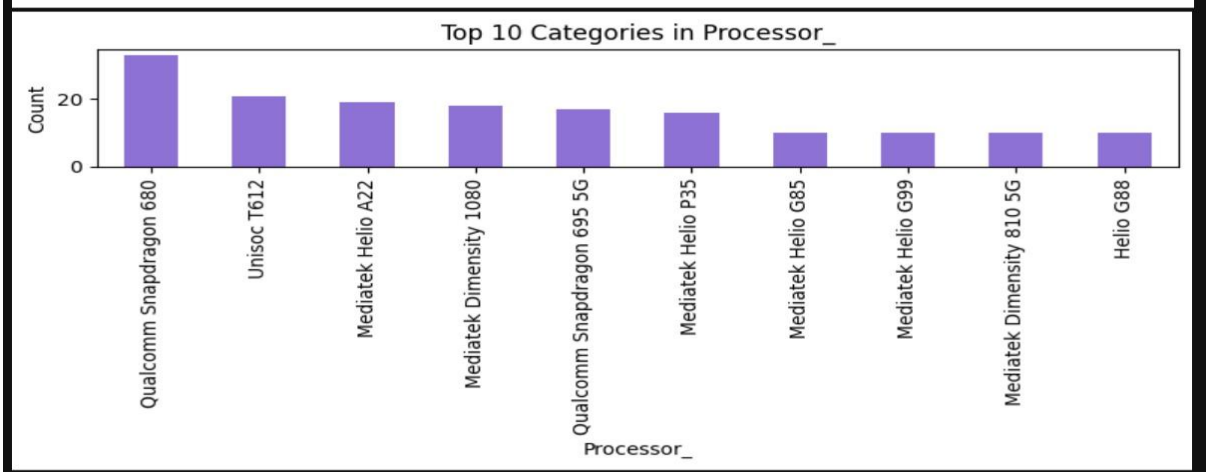
- Feature Importance Bar Chart




Count
0
2
4
6
8
10
Yellow
Royal Blue
Power Black
Black
Midnight Black
Cool Blue
Glacier Blue
Aqua Blue
Force Black
Light Green
Colour


Top 10 Categories in Rear Camera
Count
0
50
100
150
200
250
50MP
13MP
64MP
8MP
48MP
108MP
0MP
12MP
200MP
2MP
Rear Camera


Top 10 Categories in Front Camera
Count
0
20
40
60
80
100
120
140
160
16MP
5MP
8MP
13MP
32MP
0MP
50MP
12MP
60MP
2MP
Front Camera


Top 10 Categories in Processor_
Count
0
20
Qualcomm Snapdragon 680
Unisoc T612
Mediatek Helio A22
Mediatek Dimensity 1080
Qualcomm Snapdragon 695 5G
Mediatek Helio P35
Mediatek Helio G85
Mediatek Helio G99
Mediatek Dimensity 810 5G
Helio G88
Processor_

Report on EDA

The Exploratory Data Analysis revealed several key insides

- The target variable (Price) showed a right-skewed distribution, meaning most phones are moderately priced
- Features such as RAM, Battery, and Internal Storage exhibited positive correlation with Price
- Categorical features like Brand and Operating System significantly influenced pricing trends
- Outlier analysis showed that premium devices like Apple and Samsung flagships had much higher prices
- Correlation heatmap confirmed that hardware-related attributes (RAM, Battery, Storage) dominate pricing

Learning Outcome

- Learned how to clean and preprocess real-world datasets effectively.
- Understood the importance of EDA in uncovering trends and insights.
- Gained hands-on experience applying multiple regression models.
- Learned to evaluate models using R^2, RMSE, and MAE metrics.
- Understood the role of feature importance in data-driven decision-making

Recommendation and Insides

Based on the feature importance results from Gradient Boosting:

- RAM and Internal Storage have the strongest influence on price.
- Battery capacity and brand reputation are moderate contributors.
- Display size and camera quality also play roles in price determination.
- Companies can focus on these specs to optimize pricing and product positioning

Conclusion

The Gradient Boosting Regressor achieved the best performance with an R^2 score of approximately 0.89 and the lowest RMSE value. The analysis revealed that RAM, Internal Storage, and Battery Capacity are the strongest predictors of mobile phone price. This project demonstrates how machine learning can effectively assist in pricing strategy and product positioning in the smartphone market.

Reference

"Column Transformer with Mixed Types." *scikit-learn.org*.

Dishaa. "What is Feature Extraction and Feature Extraction Techniques." *analyticsvidhya*, https://www.analyticsvidhya.com/blog/2021/04/guide-for-feature-extraction-techniques/.

"GradientBoostingRegressor — scikit-learn 1.7.2 documentation." *Scikit-learn*, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html.

"OneHotEncoder — scikit-learn 1.7.2 documentation." *Scikit-learn*, https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html.

"7.3. Preprocessing data — scikit-learn 1.7.2 documentation." *Scikit-learn*, https://scikit-learn.org/stable/modules/preprocessing.html.

Zhang, Justin. "Machine Learning: Feature Selection and Extraction with Examples." *Medium*, medium.com, https://medium.com/nerd-for-tech/machine-learning-feature-selection-and-extraction-with-examples-80e3e2c2e1a1.

THANK YOU